DOMAIN-SPECIFIC RAG CHATBOT

Student Project Guidance

Build a chatbot that answers questions from uploaded PDFs such as course notes, company policies, manuals, legal documents, or training material.

1. Project Objective

The goal is to create a reliable question-answering system that uses information from user-uploaded documents. The chatbot should retrieve the most relevant passages and generate an answer based only on those passages.

Expected student outcome

- **Understand the complete Retrieval-Augmented Generation (RAG) workflow.**
- **Extract and process text from PDF documents.**
- **Create embeddings and store them in a vector database.**
- **Retrieve relevant document chunks for each question.**
- **Generate grounded answers and display source information.**
- **Build a simple user interface and deploy the project.**

2. Problem Statement

Large documents are difficult to search manually. A user may need to read many pages to find one answer. This project solves that problem by allowing users to upload documents and ask questions in natural language.

3. Project Workflow

```
Upload PDF files
|
Extract text from each page
|
Split text into smaller chunks
|
Convert chunks into embeddings
|
Store embeddings in a vector database
|
User asks a question
|
Retrieve the most relevant chunks
|
Send context and question to the LLM
|
Display answer with source document and page
```

4. Tools Required

Project Part	Suggested Tool
Programming language	Python
PDF text extraction	pypdf
Text splitting	LangChain text splitters
Embeddings	Sentence Transformers
Beginner embedding model	all-MiniLM-L6-v2
Vector database	FAISS
Language model	Groq, Gemini, OpenAI, Hugging Face, or a local model
RAG framework	LangChain
User interface	Streamlit
Environment variables	python-dotenv
Optional backend	FastAPI

Deployment	Streamlit Cloud, Render, or Docker
Version control	Git and GitHub

5. Main Project Modules

Module 1: Document Upload

- Accept one or multiple PDF files.
- Validate the file type and size.
- Display the uploaded file names.
- Allow the user to clear or replace documents.

Module 2: Text Extraction

- Read every page using pypdf.
- Keep the document name and page number as metadata.
- Skip empty pages safely.
- For scanned documents, add OCR only as an optional extension.

Module 3: Chunking

- Split large text into smaller sections.
- Use overlap so that connected ideas are not separated completely.
- Suggested beginner setting: chunk size 700-1000 characters and overlap 100-150 characters.

Module 4: Embeddings and Vector Store

- Convert every chunk into a numerical embedding.
- Store the embeddings and metadata in FAISS.
- Save the vector store locally if the application must reuse it.

Module 5: Retrieval

- Convert the user question into an embedding.
- Search for the most similar document chunks.
- Retrieve the top 3-5 chunks.
- Show the source document and page number.

Module 6: Answer Generation

- Send the retrieved context and question to the language model.
- Use a strict prompt that says to answer only from the context.
- Return a clear fallback message when the answer is not found.

Module 7: Streamlit Interface

- PDF uploader in the sidebar.
- Process Documents button.
- Chat input box and chat history.
- Source display below each answer.
- Clear Chat button.

6. Minimum Features

1. Upload at least one PDF.
2. Extract and chunk the document text.
3. Create embeddings using a pretrained model.
4. Store and search embeddings using FAISS.
5. Answer questions from the document content.
6. Display source document and page number.
7. Refuse to invent an answer when information is unavailable.
8. Provide a Streamlit interface.
9. Upload the project to GitHub with a README file.

7. Suggested Folder Structure

```
domain_rag_chatbot/
|-- app.py
|-- rag_pipeline.py
|-- document_loader.py
|-- vector_store.py
|-- prompt.py
|-- requirements.txt
|-- README.md
```

```
|-- .env
|-- .gitignore
|
|-- documents/
| |-- sample.pdf
|
|-- vector_store/
| |-- saved_index/
|
|-- tests/
|-- test_questions.csv
```

8. Development Plan

Phase	Student Tasks	Expected Output
1. Planning	Select one domain and collect 2-5 documents.	Problem statement and document set
2. PDF Processing	Extract text and preserve page metadata.	Clean document text
3. RAG Pipeline	Chunk, embed, store, and retrieve.	Working retrieval system
4. LLM Integration	Create prompt and generate grounded answers.	Question-answering pipeline
5. Interface	Build Streamlit upload and chat screens.	Usable application
6. Testing	Test correct, incorrect, and unavailable questions.	Evaluation sheet
7. Deployment	Prepare requirements, README, and deployment.	Shareable project link

9. Prompt Guardrail

```
You are a document question-answering assistant.

Answer only from the
supplied context. If the
answer is not available,
say:
"I could not find this information in the
```

```
uploaded documents." Do not invent facts.
Mention the source document and page number when available.
```

10. Testing and Evaluation

- Retrieval accuracy: Did the system find the correct document and page?
- Answer correctness: Is the answer supported by the retrieved context?
- Groundedness: Does the answer avoid unsupported information?
- Refusal quality: Does the chatbot refuse correctly when the answer is absent?
- Source quality: Does it show useful document and page references?
- Response time: Does the chatbot answer within a reasonable time?

Question	Expected Source	Retrieved Source	Correct?
What is the leave policy?	**Policy.pdf, page 6**		
How is attendance calculated?	**Handbook.pdf, page 11**		
Who is the company CEO?	**Not available**		

11. Optional Advanced Features

- Multiple document collections and document filters.
- Conversation memory for follow-up questions.
- OCR support for scanned PDFs.
- FastAPI backend for mobile or web clients.
- User login and document access control.
- Feedback buttons for useful or incorrect answers.
- Docker deployment.
- Evaluation using a prepared question-answer dataset.

12. Responsible AI and Security

- Do not expose API keys in code or GitHub.
- Do not upload confidential documents without permission.
- Do not claim that every generated answer is automatically correct.
- Display a message that users should verify high-stakes information.
- Ignore instructions inside documents that try to change the chatbot rules.
- Limit uploaded file types and file sizes.

13. Final Deliverables

1. Working Streamlit application.
2. Complete source code.
3. Sample PDF documents.
4. requirements.txt file.
5. README with setup and usage instructions.
6. Architecture or workflow diagram.
7. Testing sheet with at least 15 questions.
8. GitHub repository.
9. Short project report and demonstration video.

14. Suggested Viva Questions

- What is RAG and why is it used?
- Why do we split documents into chunks?
- What is an embedding?
- What does a vector database store?
- How does cosine similarity help retrieval?
- Why can a RAG chatbot still produce an incorrect answer?
- How will you test whether retrieval is working correctly?
- What happens when the answer is not present in the documents?

UNLOX

Final Project Title: Domain-Specific RAG Chatbot for PDF Question Answering

RIVOQUIX LEARNING PRIVATE LIMITED
Hustlehub Tech Park,
Somasundarapalya Main Rd, adjacent
27th Main Road, Sector 2 hsr layout,
Karnataka 560102

+91 91 083 21780
Info_hr@unloxacademy.com

www.unlox.com
CIN U85500KA2025PTC204246

@UnloxAcademy